Hiero Workflow App Architecture-First Response

Pre-interview task | Darshit | LFDT Mentorship 2026

Thesis. Build a **GitHub App (Probot)** as the product centre with adapter-agnostic decision logic in `packages/core`. Start with `/assign` as the first product slice, then add PR quality, lifecycle, and review modules one vertical at a time. GitHub Actions serve as a compatibility / batch adapter over the same core.

Centralise: decision logic, schemas, config validation, parity tests, adapters, releases. **Keep local:** workflow YAML, triggers, permissions, checkout, harden-runner, policy. **Never rewrite:** SDK security controls or maintainer governance.

Q1. Well-Architected Principles Applied to Hiero [4 pts K=2, R=2]

[K = 2] **Clear responsibility.** SDK repos own triggers, permissions, checkout, harden-runner, secrets, and local policy. The central GitHub App owns automation decisions, schemas, adapters, and releases. No step hides a security control or merges two governance domains.

[K = 2] **Secure by design.** Webhook HMAC-SHA256 verification at the listener boundary; installation-scoped tokens; CODEOWNERS on config; fail closed on missing or invalid config; strict command parsing.

[R = 2] **Low coupling / high cohesion.** Decision logic lives in `packages/core`; the Probot GitHub App and the Actions adapter are thin delivery layers over the same functions behaviour tested once.

[R = 2] **Observable and testable.** Every automation supports structured logs (Pino), idempotent writes via bot markers, golden fixtures, and parity tests. No module reaches production before its parity fixtures are green and an integration test covers the full pipeline.

Evolvable. New automations register in the core module registry; config schema versions with backward-compatible windows. No cross-SDK script copying.

[K = 2] **Microkernel foundation.** `packages/core` is the decision kernel; the Probot App and Actions adapter are delivery plug-ins (Richards, 2015). New modules register without modifying adapter code.

Q2. Proposed Final System Architecture [4 pts R=2, A=2]

[R = 2] [A = 2] **Eight-stage pipeline** inside the GitHub App: Listener (HMAC-SHA256 verified) → Normalizer (stable internal event model) → Router (declarative route registration) → Dispatcher (config-loaded, module-selected) → Policy Module (pure function, no I/O) → Executor (idempotent, installation-scoped GitHub writes) → Audit Logger (Pino, append-only). Config Engine feeds the Dispatcher. **SDK repos** own workflow control, security, and per-repo config; **GitHub Actions adapter** calls the same `packages/core` modules for batch / scheduled operations.

Invariant: centralise reusable decision logic; do not centralise repository control. The App reads per-repo config it never owns it.

C4 L2 container	Responsibility
SDK repos	Workflow YAML, triggers, permissions, harden-runner, checkout, concurrency, policy config.
sdk-automations (GitHub App)	Probot App (primary webhook path), Actions adapter (batch path), `packages/core` (shared modules: Assignment Policy, PR Quality Policy), schemas, fixtures, releases.
GH Actions	Compatibility / batch path: scheduled runs and SDK-owned workflows calling same core.

Q3. Architectural Boundaries and Why They Exist [4 pts R=2, E=2]

[R = 2]

Repo-local — YAML, permissions, checkout, harden-runner, concurrency, token. Remain local because these are security and maintainer-ownership controls. Moving them hides the security surface from the team accountable for it.

Shared-core — (a) Module registry and dispatcher interface, (b) shared utilities (comment builder, label writer, config loader), (c) policy modules as registered pure functions. Central because duplication creates maintenance cost and behavioural drift.

Policy — Labels, team handles, docs links, limits, enabled automations. Protected local config so SDK-specific choices do not become code forks in the shared repo.

Hosted-app — Boundary between the open internet (GitHub webhook delivery) and the trusted internal pipeline. No module bypasses the listener; no module calls GitHub directly — all writes flow through the Executor.

Three-bucket migration filter. Every code unit is classified before moving: **(1)** shared-core candidate common decision logic, moves to `sdk-automations`; **(2)** repo-local boundary workflow YAML and security controls, permanent; **(3)** repo-specific policy expressed as protected config contract, never a hardcoded central module. Any unit that cannot be cleanly classified without resolving policy differences between Python and C++ stays local until the behaviour map is complete.

[E = 2] **Cost of crossing each boundary: Repo-local** → central: hides permissions from the accountable team; one compromised workflow YAML can escalate across all installed repos. **Core** → per-repo: creates hidden code forks; policy drift defeats centralisation. **Policy** → hardcoded: forces C++ to adopt Python defaults (or vice versa), breaking SDK-specific governance. **App hosting skipped**: loses real-time webhook response; batch-only Actions cannot handle interactive commands like `/assign`.

Q4. Tooling Choices [6 pts R=2, A=2, E=2]

[R = 2]

Area	Recommendation and justification
Caller runtime	**GitHub App (Probot) first.** Real-time webhook processing for `/assign` and PR checks. Actions as compatibility / batch path.
Hosting	**Containerised Node.js / TypeScript** (Cloud Run or Railway) with Redis for delivery dedup and config cache; blue-green rollout; decision gated before Phase 1.
Core language	**Node.js / TypeScript + Octokit.** Type-safe across the pipeline; module interfaces enforced at compile time; bundles natively as an Action artefact.
Shared logic	`packages/core`**.** Adapter-agnostic modules. Probot and Actions adapter call the same functions — behaviour tested once, delivered via two paths.
Config	**JSON / YAML + Zod schema.** Zod validates at runtime and generates TS types from the schema. Tries `.json`, `.yml`, `.yaml`; fails closed on invalid.
Verification	**Unit + mock + fixtures + parity.** Each layer catches a different failure class: logic, API contracts, or real-world call paths.
Security ops	**Harden-Runner, CODEOWNERS, SHA pins, Dependabot, rollback docs.** Governance gate, not optional extras.

[A = 2] [E = 2] Every choice is justified by what it *prevents*. Node.js bundles natively; JSON Schema enforces a contract. Tooling that cannot be pinned, audited, or rolled back is not acceptable in a security-sensitive automation path.

Q5. How This Solution Builds Iteratively [2 pts A=2]

[A = 2]

1. **Architecture + App shell first.** Webhook listener, config engine, event routing, and fail-closed validation before any policy module is attached.
2. **`/assign` as the first product slice.** Forces the full App path: comment parsing, config load, policy evaluation, GitHub write, and audit. Bounded scope.
3. **`/assign` guards and GFI cap.** Skill prerequisites, open-assignment limit, Good First Issue graduation cap — parity-tested against C++ and Python behaviour.
4. **PR quality module.** DCO, GPG, merge conflict, and issue-link checks with dashboard comments and status label management.
5. **Review-sync and lifecycle modules** follow once the App shell is trusted and `/assign` parity is proven.
6. **Retire local code only after three gates:** parity tests green, pilot evidence matches expectations, and maintainer sign-off.

Q6. Malicious Pull Request Risks and Safeguards [10 pts K=4, R=3, A=3]

[K = 4] Highest-risk design area. The architecture assumes a malicious contributor controls: PR title / body, branch name, changed files, artefacts from untrusted code, and workflow / config changes in the PR branch. The App **always loads config from the default branch** via the GitHub API never from the PR head so attackers cannot inject policy overrides through a malicious PR.

Risk	Safeguard
Forged / replayed webhook	HMAC-SHA256 signature verification at the listener; delivery-ID deduplication via Redis `SET NX` (24h TTL); on store unavailability, fail closed.
Config / policy tampering	CODEOWNERS on `.github/hiero-automation.*`; schema-validate every invocation; reject unknown fields; fail closed on missing config (no silent defaults).
Command injection	Strict regex parser (`/^\s*assign\s*$/i`); no argument pass-through; actor-type and bot guards before processing.
Token / secret exfiltration	Installation-scoped tokens; harden-runner in SDK workflows; restrict / audit egress; never checkout PR head in privileged jobs.
Action / dependency compromise	Pin all actions by full commit SHA; bundle central actions; Dependabot and `npm audit` continuously.
Over-broad permissions	GitHub App requests minimum scopes: Issues (write), Pull Requests (read), Contents (read). Installation-scoped tokens; no org-wide access.
Bot loops / broad writes	Actor / bot guards, idempotent bot markers, scoped label allowlists, concurrency keys, rollback by disabling the App installation or pinning back a SHA.

[R = 3] [A = 3] **Systemic safeguards:** validate actor type, event type, repo, PR number, label allowlists, and config schema *before* acting — unknown config fails closed. All writes are idempotent and scoped to known bot markers only. Config is loaded from the default branch via GitHub API (never from PR head); rate-limit handling with exponential back-off centralised in the Executor — no module retries independently, preventing duplicate write risk.
Fail-closed default: missing or malformed config → error logged, no state mutations occur. **Partial-write recovery:** the Executor checks current GitHub state before any retry — a label already present is not re-applied; a comment with an existing bot marker is not re-posted. Per-entity concurrency locks (Redis `SET NX` per issue / PR) serialise concurrent webhooks.

Q7. Configuration Standardisation vs. User Input [6 pts K=2, R=2, A=2]

[K = 2] [R = 2]

Standardise centrally	Allow per-repo (with guardrails)
Config schema version, automation names, command syntax, bot-comment markers, idempotency logic, error categories, fail-closed defaults, release compatibility.	Label names, team handles, docs links, assignment limits, GFI graduation cap, skill hierarchy, enabled automations, schedules, repo-specific message links.

When Python and C++ diverge on a value, that value is repo-specific policy never hardcoded into `packages/core`. Core handles the decision *shape*; the per-repo config supplies the policy *parameter*. Concrete example: C++ sets `max_good_first_issue_completions: 5` and `max_assignments: 1` to enforce a graduation path; Python omits these fields entirely, using permissive defaults. The same `runAssign` code path serves both — no fork required.

[A = 2] **Rule of thumb:** common behaviour → core module; repo policy → protected config validated by the Zod schema (e.g. C++ sets `max_good_first_issue_completions: 5`, Python omits it); one-off workflow / security constraint → keep strictly local. Config expresses policy, not code.

Q8. Community Leverage Plan [10 pts K=3, R=3, A=4]

[K = 3] **Public issue backlog.** Architecture becomes a contribution surface covering behaviour mapping, fixture collection, docs, config examples, golden tests, canary workflows, security review, and SDK adoption guides useful work that does not require maintainers to triage risky rewrites.

[R = 3] **Skill-based labels:** `good-first-issue` (docs / fixtures) · `beginner` (tests / samples) · `intermediate` (adapters / config) · `advanced` (security pilots / C++ investigation).

[A = 4] **RFC per automation migration.** Each RFC states: reusable logic, repo-local boundary, policy config, fixtures, rollback plan, and owner. No production behaviour change without dry-run proof, parity tests, a security checklist, and maintainer approval. Small, reviewable PRs only.

Work package	Good community issue shape
Behaviour mapping	Compare Python and C++ for one automation; record common logic, policy differences, fixtures, and unknowns.
Fixture / parity tests	Convert real examples to golden cases so central behaviour is proved before writes.
Docs / examples	Caller workflow examples, config samples, rollback instructions, security checklists.
Canary support	Run dry-run pilots, collect logs, identify false positives, file follow-up issues.
Security review	Threat-model `pull_request_target`, artefact flows, permissions, installation scopes, and release pinning.

Built-in contributor progression. The `/assign` system itself creates a structured onboarding funnel: skill labels (`good-first-issue` → `beginner` → `intermediate`) with prerequisite gates and a GFI graduation cap teach contributors to grow through progressively harder tasks the App does not just automate maintainer work, it actively develops the contributor pipeline.

Demos and office hours. Publish reproduction steps so contributors can verify call paths independently. SDK maintainers review for behaviour; central maintainers review for schema, release, and security.

First safe batch (open immediately): (1) `/assign` behaviour matrix (Python / C++ guard comparison); **(2)** `/assign` golden fixtures and parity tests; **(3)** full-SHA pinning policy doc; **(4)** schema compatibility and config versioning contract; **(5)** SDK caller adoption checklist for maintainers. Issues to defer: full C++ lifecycle migration, review-sync state machine, and production App hosting at scale — all require earlier gates to close first.

Q9. Architecture vs. Immediate Development [4 pts R=2, E=2]

[R = 2]

Option	Risk	Assessment
Skip architecture	High	Replicates existing problem: independent evolution, inconsistent security, no parity baseline.
Full freeze	Medium	Delays reversible work unnecessarily; fixtures, schema, and dry-run adapters carry no risk.
2-week arch + reversible work	**Low**	**Recommended.** Architecture gates production writes; reversible work runs in parallel.

[E = 2] **Allocation: Wk 1** — App shell skeleton, boundary definitions, threat model, config schema. **Wk 2** — `/assign` behaviour maps, guard matrix, rollback plan, community backlog.
Safe to begin now: App shell, config engine, `/assign` module, parity fixtures, docs. **Must wait:** upstream production enablement, SDK security control changes.
Phase 0 already complete: Python fork canary (run proof), central repo CI (CI proof), and Probot adapter with 68 passing tests across three workspaces (core: 55, Probot: 10, Actions adapter: 3).

References: arc42.org · GitHub Actions secure-use docs · StepSecurity Harden-Runner · LFDT Issue #73 · C++ Issue #1627 · Richards, *Software Architecture Patterns* (O'Reilly, 2015)